

Random maze generation by recursive division, and shortest-path search to the exit

Mohamed Ali Tewfik Hamlil

M1 Artificial Intelligence, Université Grenoble Alpes
Algorithmic Problem Solving — APP1

September 12, 2026

Abstract

A rectangular area of $W \times H$ cells is laid out as a maze by recursive division: a full wall across the longest dimension, exactly one door in it, and the same rule inside each of the two halves. This report states that as a graph problem, shows that the requirements force the maze to be a spanning tree of the grid graph, and derives a generation algorithm together with the data structure it runs on and a proof that what it returns is that spanning tree. Giving every door unit weight makes the length of a route the number of doors crossed, which is what the shortest-path search of Section 4 minimises and what the Manhattan and Euclidean distances are compared against as heuristic guidance. A Python implementation checks the claims on generated instances, and the closing section gives the size of the structure and the cost of both algorithms.

Contents

1	Problem formulation	2
1.1	Statement	2
1.2	From a drawing of walls to a graph	2
1.3	What the specification forces E to be	3
1.4	The two problems, restated on the graph	3
1.5	Conventions	4
1.6	Scope of the deliverable	4
2	Generating the maze into a graph	4
2.1	What has to be stored	4
2.2	The cell array: two links per cell	4
2.3	The region tree	5
2.4	Starting full, then deleting	5
2.5	One partition step	6
2.6	Why a one-cell-wide region is not a special case	7
2.7	The algorithm	7
2.8	Why the result is a spanning tree	7
2.9	The graph that comes out	9
3	Encoding the distance between adjacent cells as 1	9
4	Shortest path to the exit, with distance as heuristic guidance	9
5	Testing with a Python implementation	9
6	Complexity and analysis of the results	9

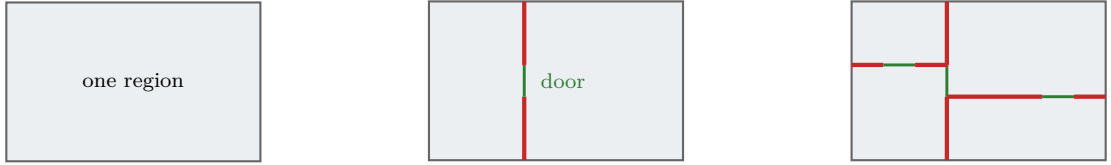
1 Problem formulation

1.1 Statement

A rectangular area of $W \times H$ unit cells is to be laid out as a maze. The layout must be drawn at random, so that W and H do not determine it; the area has a single exit on its boundary; and it is entered at an arbitrary interior cell. Two problems follow, and both are to be settled algorithmically before any implementation and then analysed for cost:

1. given the width and the height of the initial area, generate a maze;
2. find the path from any position in the maze to the exit, which the specification states is unique and hence shortest.

The construction is prescribed. Divide the allocated space with a full wall across its longest dimension, leave exactly one door in that wall, and apply the same rule to each of the two spaces this creates. Figure 1 is the entire rule; everything the report derives follows from it.



- (a) A rectangular region, every cell reachable from every other. (b) One full wall across the longest side, pierced by exactly one door. (c) The same rule applied inside each of the two halves.

Figure 1: The construction rule. Each application turns one region into two regions of the same kind joined by a single door, which is what makes this a divide-and-conquer problem and the decomposition a binary tree.

1.2 From a drawing of walls to a graph

A maze drawn on paper is a picture of walls; what an algorithm can generate, store and search is a graph. The translation is the first decision of this report, and every later section rests on it.

Definition 1 (Area, cells, grid graph). Let $W, H \in \mathbb{N}^*$ be the width and the height of the allocated area, measured in unit cells. The cells are

$$V = \{(x, y) \in \mathbb{N}^2 : 0 \leq x < W, 0 \leq y < H\}, \quad |V| = WH,$$

and two cells are *adjacent* when they share a side, that is when $|x - x'| + |y - y'| = 1$. The *grid graph* $G_{\text{grid}} = (V, E_{\text{grid}})$ has the cells as vertices and the adjacent pairs as edges, so

$$|E_{\text{grid}}| = \underbrace{H(W - 1)}_{\text{horizontal}} + \underbrace{W(H - 1)}_{\text{vertical}} = 2WH - W - H.$$

Definition 2 (Maze). A *maze* on that area is a spanning subgraph $M = (V, E)$ with $E \subseteq E_{\text{grid}}$. An edge of E is a *door*: the two cells it joins can be walked between. A pair of $E_{\text{grid}} \setminus E$ is a *wall segment*. One distinguished boundary cell v_{exit} is the exit.

Figure 2 puts the two readings of the same object side by side. The vertex set is fixed by W and H alone; generating a maze therefore means nothing more than *choosing which subset of E_{grid} to keep*.

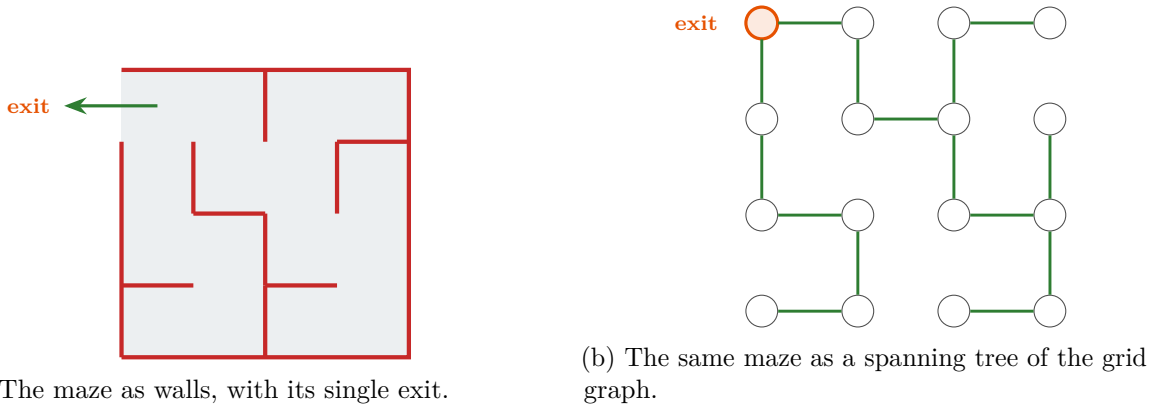


Figure 2: A maze is not a picture but a graph. Cells become vertices, doors become edges, and a wall is simply an edge of the grid graph that is absent. The two panels show the same 4×4 maze.

1.3 What the specification forces E to be

Three requirements follow from the statement.

- (R1) **Every cell reaches the exit.** The entry position is arbitrary, so any cell may be a starting point and each must have a route out: M is connected.
- (R2) **There is exactly one exit.** v_{exit} is a single cell on the boundary of the area, and the outer wall is closed everywhere else.
- (R3) **The route out does not fork.** The path to the exit is required to be unique, hence shortest; a cycle in M would give two distinct simple routes between the cells it passes through, so M is acyclic.

Connected and acyclic on a fixed vertex set is a spanning tree, which settles the shape of the answer before any algorithm is written.

Proposition 1 (The object to produce). *A subgraph $M = (V, E)$ of G_{grid} satisfies (R1) and (R3) if and only if it is a spanning tree of G_{grid} . It then has $|E| = WH - 1$ doors, and between any two cells there is exactly one simple path, which is therefore also a shortest one.*

That last clause is what makes the second problem well posed: asking for *the* path out is only meaningful because there is one. It is also the lever for the rest of the report — Section 2 shows that the construction rule produces such a tree *by construction* rather than by checking afterwards.

1.4 The two problems, restated on the graph

Problem 1 — Generation.

Given W , H , an exit cell v_{exit} and a source of random bits, produce a spanning tree M of G_{grid} , drawn at random so that no two runs need agree, together with the structure that records how it was built.

Problem 2 — Escape.

Given M and any start cell $s \in V$, return the unique simple path from s to v_{exit} .

Problem 1 is solved in Section 2. Problem 2 needs a notion of length before it can be measured: Section 3 gives every door weight 1, and Section 4 runs the shortest-path search that weight makes meaningful, guided by the Manhattan and Euclidean distances.

1.5 Conventions

Symbol	Meaning
W, H	width and height of the area, in cells
$n = WH$	number of cells, the size of the input
(x, y)	cell in column x (left to right) and row y (top to bottom)
$G_{\text{grid}}, E_{\text{grid}}$	the grid graph and its edges: every adjacency, all doors open
$M = (V, E)$	the maze produced: a spanning tree of G_{grid}
v_{exit}	the single exit cell, on the boundary
$[a_0, a_1] \times [b_0, b_1]$	a region: columns $a_0..a_1$, rows $b_0..b_1$

1.6 Scope of the deliverable

The specification fixes the contents and not only the topic: a data structure with a statement of what it holds, an algorithm for each of the two problems, the size of that structure and the cost of both algorithms. The table below is the report's own checklist against that list.

Required	Where	State
Data structure, and what it stores	Section 2	done
Maze generation algorithm	Section 2	done
Cost of a door, and what a path costs	Section 3	to write
Path-finding algorithm	Section 4	to write
Implementation (optional in the specification)	Section 5	to write
Size of the structure, complexity of both algorithms	Section 6	to write

2 Generating the maze into a graph

Section 1 fixed the target: a spanning tree of G_{grid} , drawn at random. This section builds one. The structure comes first, because the algorithm is written against it, and the guarantee comes last, because it is the construction rule that supplies it.

2.1 What has to be stored

Two different things are worth keeping, and confusing them is the usual way to end up storing neither well.

The maze itself — which doors exist.

A subset of E_{grid} . It is what a walker needs, and the only thing needed to answer Problem 2.

The way it was built — the region decomposition.

Which wall was put where, and in what order. It is the binary tree the divide-and-conquer rule produces for free, and the object whose depth the cost analysis of Section 6 is written in terms of.

2.2 The cell array: two links per cell

The doors are held in a $W \times H$ array T whose entry $T[x][y]$ is one cell record holding two links,

$$T[x][y] = \langle P_0, P_1 \rangle,$$

where P_0 points at the cell $(x+1, y)$ to its right and P_1 at the cell $(x, y+1)$ below it, and where a link set to NIL means *no door there*. Figure 3 draws the arrangement.

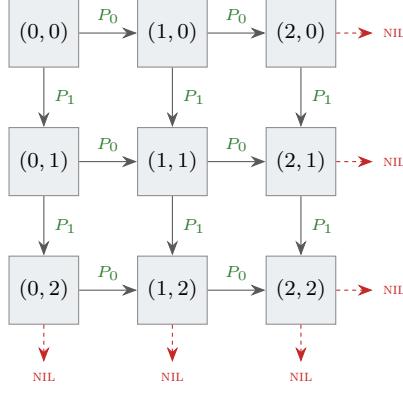


Figure 3: The cell record. Every cell stores two links, P_0 to the cell on its right and P_1 to the cell below it; the cells of the last column and the last row carry NIL in the direction that leaves the area. Each edge of G_{grid} is owned by exactly one cell, so the $2WH - W - H$ edges are held without a single one stored twice, and the four-neighbour view a search wants is read back in $O(1)$ from the left and upper neighbours' own links.

Two links, not four, is the one design decision in the structure. Every edge of G_{grid} joins a cell to the cell on its right or the cell below it, never to a third kind of neighbour, so *ownership by the upper-left endpoint* covers the whole edge set exactly once: the $2WH - W - H$ possible doors are held in $2WH$ slots with no edge duplicated, and the last column and last row are where the unused slots sit.

Remark. The four-neighbour record — a node holding its coordinates and an array *Neighbour*[4] — is the convenient *view* for a search, which wants all ways out of a cell at once. It is not a second structure: the neighbours of (x, y) to the west and north are recovered in $O(1)$ from $T[x-1][y].P_0$ and $T[x][y-1].P_1$. Materialising it costs twice the memory and buys no information, so this report stores two links and reads four.

2.3 The region tree

The second structure is a binary tree, one node per call of the recursion:

- an **internal node** records one cut — its orientation t , the boundary j the wall sits on, the position i of its single door — and has the two regions the cut created as its children;
- a **leaf** is a region reduced to one cell.

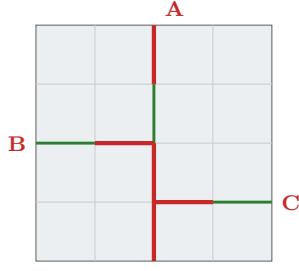
Figure 4 shows the first three cuts of a 4×4 area beside the top of the tree they build. Since every cut creates exactly one door, internal nodes and doors are in bijection: a tree with WH leaves has $WH - 1$ internal nodes, which is the $|E| = WH - 1$ of Proposition 1 arrived at from the other side.

2.4 Starting full, then deleting

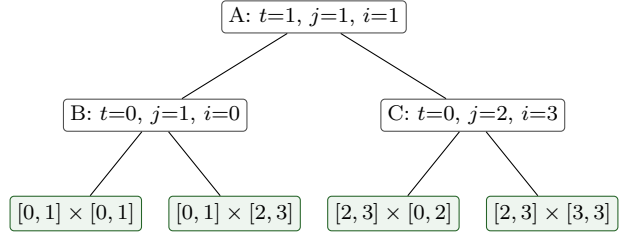
The algorithm starts from T with *every* link set: that is G_{grid} itself, all doors open, every cell reachable from every other. Generation only ever deletes. This is the direction that makes the invariant cheap to state and to keep:

At every call, the region handled is connected in T , and no edge of T joins that region to anything outside it that the caller did not put there.

It holds at the root because G_{grid} is connected, and Figure 5 is one step of preserving it.



(a) The first three cuts of a 4×4 area.



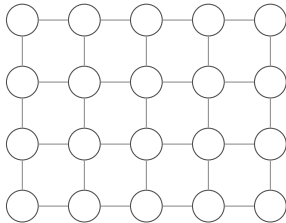
(b) The top of the region tree it builds. The four boxes at the bottom are regions still to be cut, not cells.

Figure 4: Cuts and tree nodes are in bijection. An internal node records the orientation t , the wall position j and the door position i that made the cut, and its two children are the two regions the cut created; a leaf is a single cell. Carrying this 4×4 example down to its leaves gives 16 leaves and 15 internal nodes — one per door, matching the $WH - 1$ edges of Proposition 1.

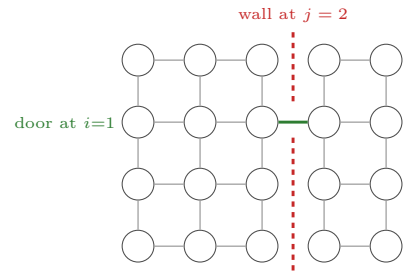
2.5 One partition step

A call handles a region $[a_0, a_1] \times [b_0, b_1]$. If the region is a single cell there is nothing to cut and the call returns a leaf. Otherwise:

1. **Choose the orientation t .** The rule puts the wall across the longest dimension, so $t = 1$ (a vertical wall, cutting the column range) when $a_1 - a_0 > b_1 - b_0$, $t = 0$ (a horizontal wall, cutting the row range) when it is smaller, and a random bit decides a tie. Because the comparison is strict, the dimension chosen is the one of *strictly* greater extent, and a tie is only possible when the two are equal; a dimension of extent 0 is therefore never cut unless both are 0, which is the base case. This is what keeps the range $[a_0, a_1 - 1]$ that j is drawn from non-empty, and it is the whole of why the recursion is total.
2. **Choose the wall position j ,** uniformly among the interior boundaries of the chosen dimension, **and the door position i ,** uniformly along the wall.
3. **Delete every crossing but one.** Walk k along the wall and delete the edge it crosses at k , skipping $k = i$.
4. **Recurse** on the two regions, and hang them under the node recording $\langle t, j, i \rangle$.



(a) On entry the region is the full grid graph on $[a_0, a_1] \times [b_0, b_1]$: connected, and full of cycles.



(b) On exit every edge crossing column boundary j is gone except the one at row i .

Figure 5: One partition step, with $t = 1$ (a vertical wall). The loop walks k over the rows $b_0..b_1$ of the region and deletes the edge $(j, k) - (j+1, k)$ for every $k \neq i$. The two halves stay connected internally, and the single surviving edge is the only way between them — which is exactly the induction step of Proposition 2.

2.6 Why a one-cell-wide region is not a special case

A region reduced to a single row or a single column is where a recursion of this shape is expected to come apart, and it is worth following one through rather than trusting the rule. Take $[2, 2] \times [0, 3]$, four cells in a column. Then $d_x = 0$ and $d_y = 3$, so $t = 0$ and the wall cuts the row range, with j drawn from $[0, 2]$ — non-empty, as it must be. The door is drawn from $[a_0, a_1] = [2, 2]$, a range of one value, so $i = 2$ is forced. The loop then runs for $k = 2$ alone, finds $k = i$, and stops. *No edge is deleted.*

Figure 6 is that call. The corridor is not an exception to the partition step; it is the partition step at its extreme, where the wall has one crossing and the door is obliged to sit on it. The recursion walks the corridor down to its leaves, doing nothing at each level, and cannot disconnect it.

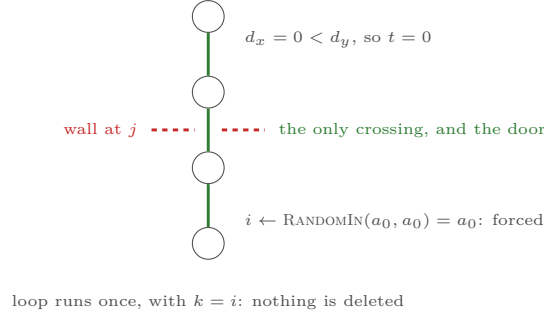


Figure 6: The degenerate region, where a recursion of this shape is usually expected to fail. A region one cell wide is cut across its other dimension, so the wall exists and the two halves are non-empty; but the wall has a single crossing edge, and the door is forced onto it because i is drawn from a range of one value. The loop body therefore never runs, the corridor descends to its leaves with every door intact, and no call can disconnect it. The same argument with the roles of x and y exchanged covers a region one cell high.

Stopping at such a region instead — returning a corridor as one leaf — is a legitimate optimisation and saves the $2n - 1$ calls a corridor of n cells costs. It is not taken here. The recursion is $\Theta(WH)$ calls either way, so the saving is a constant factor, while a corridor leaf would hold $n - 1$ doors that no internal node records, breaking the bijection of Section 2 on which the leaf-and-node counts, and the traversal of Section 4, both rest.

2.7 The algorithm

`DeleteDoor()` clears one link: deleting the edge $(j, k) - (j+1, k)$ sets $T[j][k].P_0 \leftarrow \text{NIL}$, and deleting $(k, j) - (k, j+1)$ sets $T[k][j].P_1 \leftarrow \text{NIL}$. Ownership by the upper-left endpoint means each deletion touches one slot and no lookup is needed to find it.

The exit plays no part in generation. Proposition 1 makes every cell reachable from every other, so any boundary cell may be declared v_{exit} afterwards, and any interior cell is an admissible start.

2.8 Why the result is a spanning tree

Proposition 2 (Correctness of Algorithm 1). *On a region of m cells, CarveRegion leaves the subgraph of T induced on that region connected and acyclic, with exactly $m - 1$ doors. In particular GenerateMaze returns a spanning tree of G_{grid} .*

Proof. By induction on m . If $m = 1$ the region is one cell: no edges, connected and acyclic, $m - 1 = 0$ doors, and the call returns without touching T .

Let $m > 1$. The orientation rule picks a dimension of extent at least 1 (Section 2.6), so j and the two sub-regions exist and are non-empty; call them R_ℓ and R_r , with $m_\ell + m_r = m$ cells.

Algorithm 1: Maze generation by recursive division.

Input: $W, H \geq 1$, the width and height of the area in cells.

Output: T , the cell array holding a spanning tree of G_{grid} ; r , the root of the region tree that built it.

```
1 Function GenerateMaze( $W, H$ ):
2    $T \leftarrow \text{FullGridGraph}(W, H);$                                 // all doors open
3    $r \leftarrow \text{CarveRegion}(T, [0, W - 1], [0, H - 1]);$ 
4   return ( $T, r$ );

5 Function CarveRegion( $T, [a_0, a_1], [b_0, b_1]$ ):
6   if  $a_0 = a_1$  and  $b_0 = b_1$  then return Leaf( $a_0, b_0$ );
7    $t \leftarrow \text{SplitOrientation}(a_1 - a_0, b_1 - b_0);$ 
8   if  $t = 1$  then
9      $j \leftarrow \text{RandomIn}(a_0, a_1 - 1);$                         // wall between columns  $j$  and  $j + 1$ 
10     $i \leftarrow \text{RandomIn}(b_0, b_1);$                             // the one door, on row  $i$ 
11    for  $k \leftarrow b_0$  to  $b_1$  do
12      if  $k = i$  then continue;
13      DeleteDoor( $T, (j, k), (j + 1, k)$ );
14     $\ell \leftarrow \text{CarveRegion}(T, [a_0, j], [b_0, b_1]);$ 
15     $r \leftarrow \text{CarveRegion}(T, [j + 1, a_1], [b_0, b_1]);$ 
16  else
17     $j \leftarrow \text{RandomIn}(b_0, b_1 - 1);$                         // wall between rows  $j$  and  $j + 1$ 
18     $i \leftarrow \text{RandomIn}(a_0, a_1);$                             // the one door, on column  $i$ 
19    for  $k \leftarrow a_0$  to  $a_1$  do
20      if  $k = i$  then continue;
21      DeleteDoor( $T, (k, j), (k, j + 1)$ );
22     $\ell \leftarrow \text{CarveRegion}(T, [a_0, a_1], [b_0, j]);$ 
23     $r \leftarrow \text{CarveRegion}(T, [a_0, a_1], [j + 1, b_1]);$ 
24  return CutNode( $t, j, i, \ell, r$ );

25 Function SplitOrientation( $d_x, d_y$ ):
26   if  $d_x > d_y$  then return 1;
27   if  $d_x < d_y$  then return 0;
28   return RandomBit();
```

Every edge of the region that crosses the wall does so at exactly one position k along it, and the loop deletes all of them but the one at $k = i$. No edge internal to R_ℓ or to R_r is touched, so the two recursive calls receive their regions with every internal door still open and, by the induction hypothesis, return them connected, acyclic and carrying $m_\ell - 1$ and $m_r - 1$ doors.

The only remaining edge between the two is the door at i . Joining two disjoint trees by a single edge yields a tree: it is connected, since any two cells are joined inside their own half or by crossing that one door, and acyclic, since a cycle would have to cross the wall an even, hence non-zero, number of times while only one crossing exists. The door count is $(m_\ell - 1) + (m_r - 1) + 1 = m - 1$.

Applying this to the root region gives a connected acyclic spanning subgraph of G_{grid} on all WH cells: a spanning tree, with $WH - 1$ doors. $\square$

The guarantee is therefore structural. Nothing is checked after the fact, no attempt is ever rejected and retried, and the uniqueness of the route out — the whole of Problem 2 — is a consequence of the shape of the recursion rather than of a property that had to be tested.

2.9 The graph that comes out

Three readings of the one result, none of them a separate copy:

View	Held as	Used by
Doors, compactly	the array T of $\langle P_0, P_1 \rangle$ records	generation, and storage
Adjacency	the four-neighbour view read off T	the search of Section 4
How it was built	the region tree rooted at r	the cost analysis of Section 6

Figure 2b is the output of one 4×4 run, whose first three cuts are the ones drawn in Figure 4a: 15 doors on 16 cells, one path between any two of them. What Section 3 adds is a length on those doors, so that “the path out” becomes a quantity a search can minimise.

3 Encoding the distance between adjacent cells as 1

To be written.

4 Shortest path to the exit, with distance as heuristic guidance

To be written.

5 Testing with a Python implementation

To be written.

6 Complexity and analysis of the results

To be written.